Workshop: CI/CD for Hosting Web Applications in **AWS**

Infofarm

0.1 INTRODUCTION

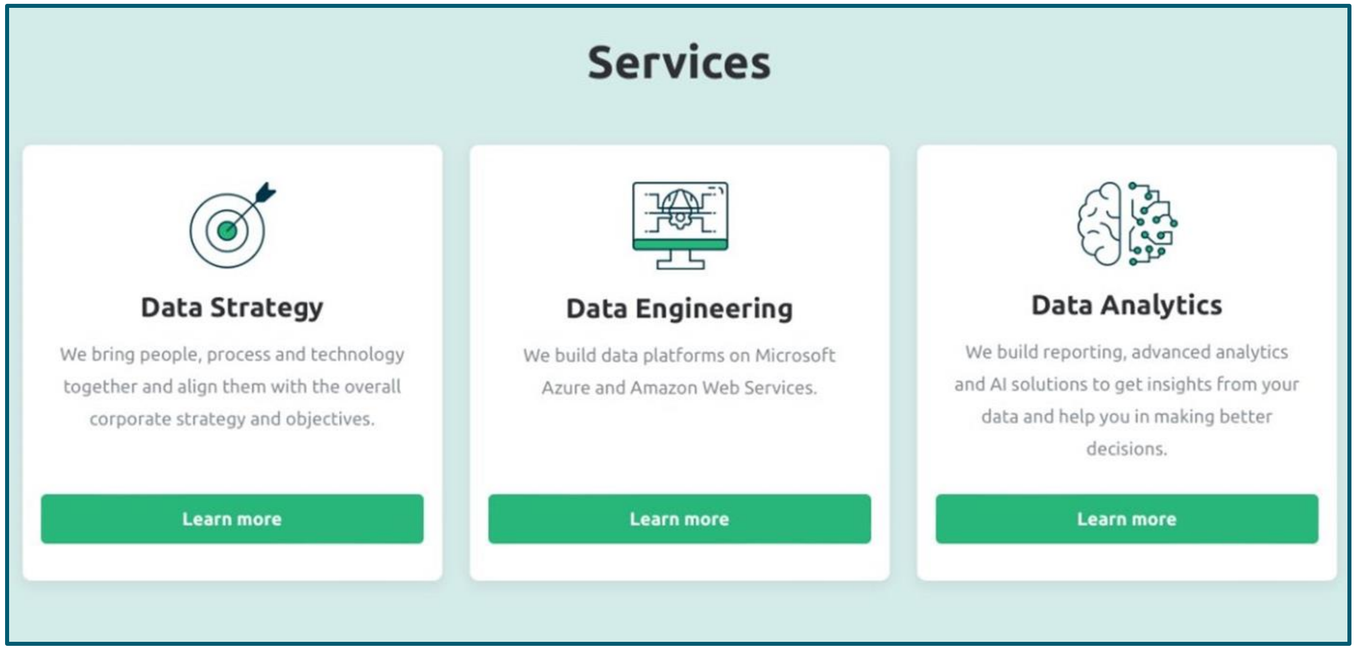
THE SPEAKERS

- AWS-native Data Engineer
- PXL Alumnus (Applied IT)
- Worked for LCM, Lineas, Digitaal Vlaanderen, Eduard

Infofarm
DATA CULTIVATORS

0.2 INTRODUCTION

THE COMPANY



- 26 consultants
- Kontich - Merelbeke - Hasselt
- Deel van Xplore & Cronos Group

Infofarm
DATA CULTIVATORS

PRESENTATION CONTENTS

STRUCTURE OF THE WORKSHOP

BUILDING & PUSHING CODE → **DEPLOYING INFRASTRUCTURE** → **AUTOMATING DEPLOYMENTS**

CI + CD = DEVOPS

Infofarm
DATA CULTIVATORS

0
Lineas Case

Standardizing web-based application hosting

0.1 PROBLEM STATEMENTS

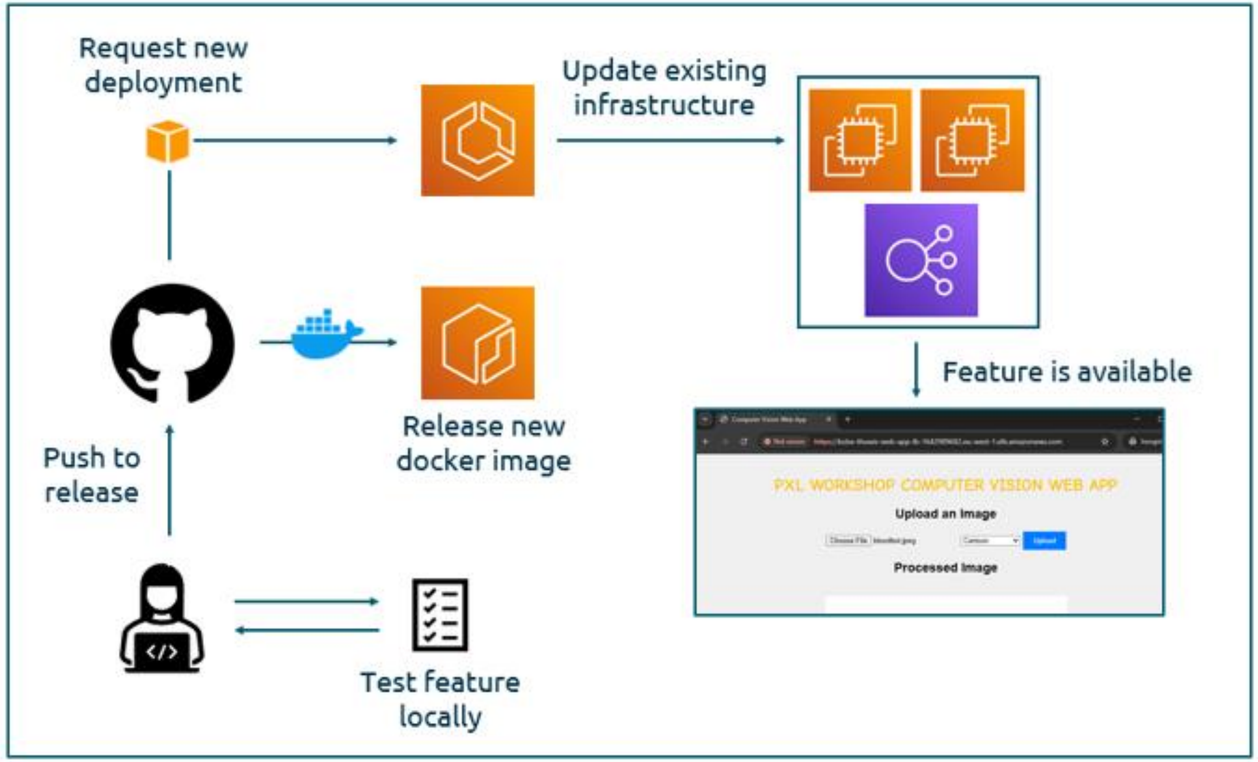
CONCRETE ISSUES

- Scalability of existing API’s
- Rapid realisation of PoC’s
- Cloud first & cost efficient

Infofarm
DATA CULTIVATORS

0.2 SOLUTION OVERVIEW

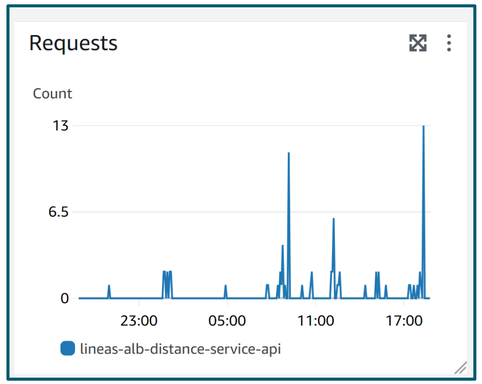
HIGH-LEVEL ARCHITECTURE

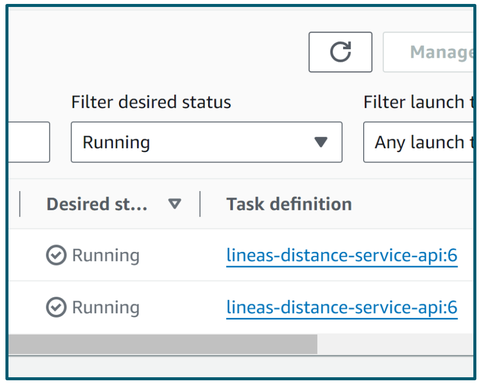


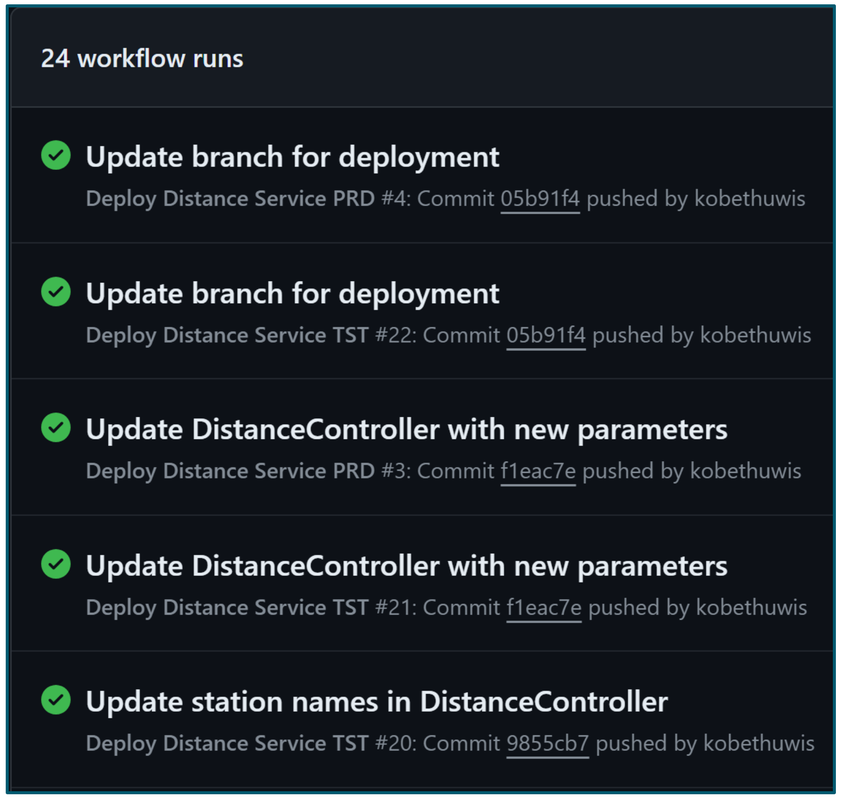
- Containerized & cloud-agnostic development
- Blue/green deployments
- Horizontally & vertically scalable

0.3 KEY ADVANTAGES

IMPACT & BENEFITS







- Transparency & traffic monitoring
- Streamlined dev process
- High availability & cost-efficient

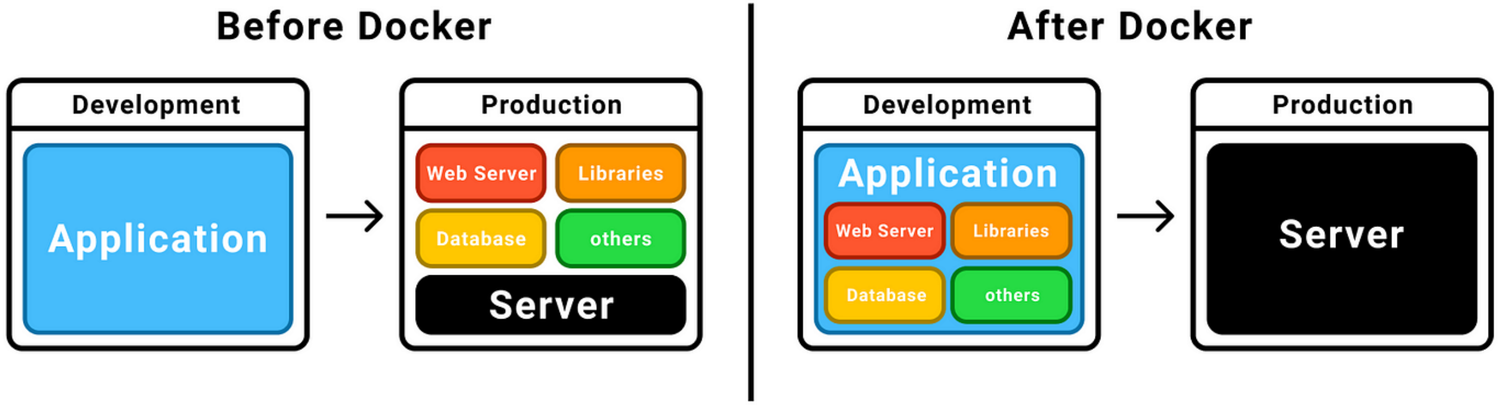
Infofarm
DATA CULTIVATORS

1
Building & pushing code

Continuous Integration (CI)

1.1 CONTAINERIZATION

THE WHY AND HOW

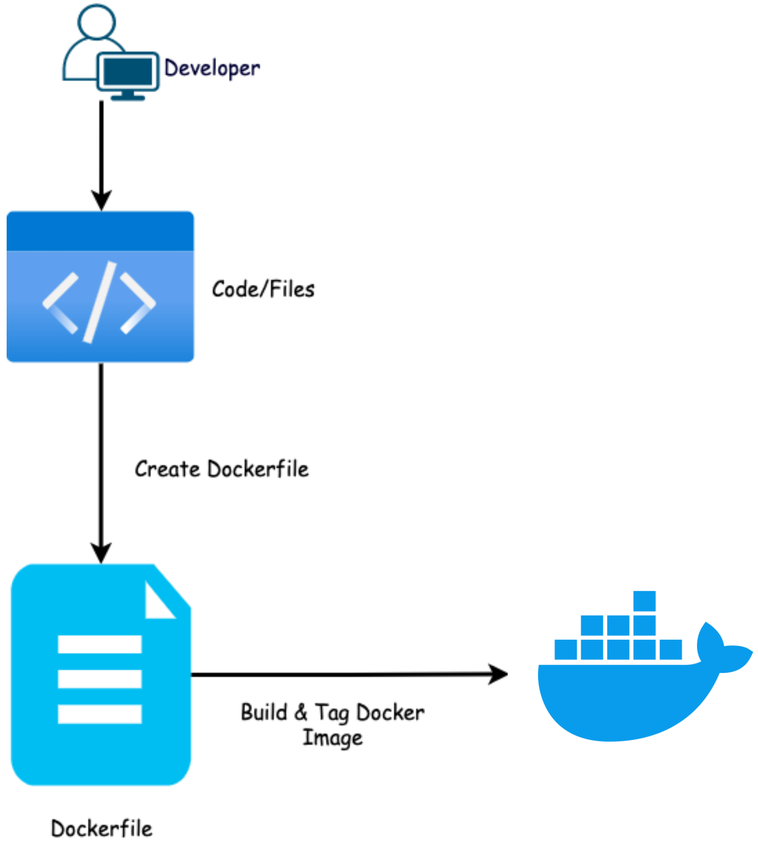


- Portable & consistent code
- Lightweight (CPU/Mem)
- Consistent build/test cycle

Infofarm
DATA CULTIVATORS

1.2 USING DOCKER

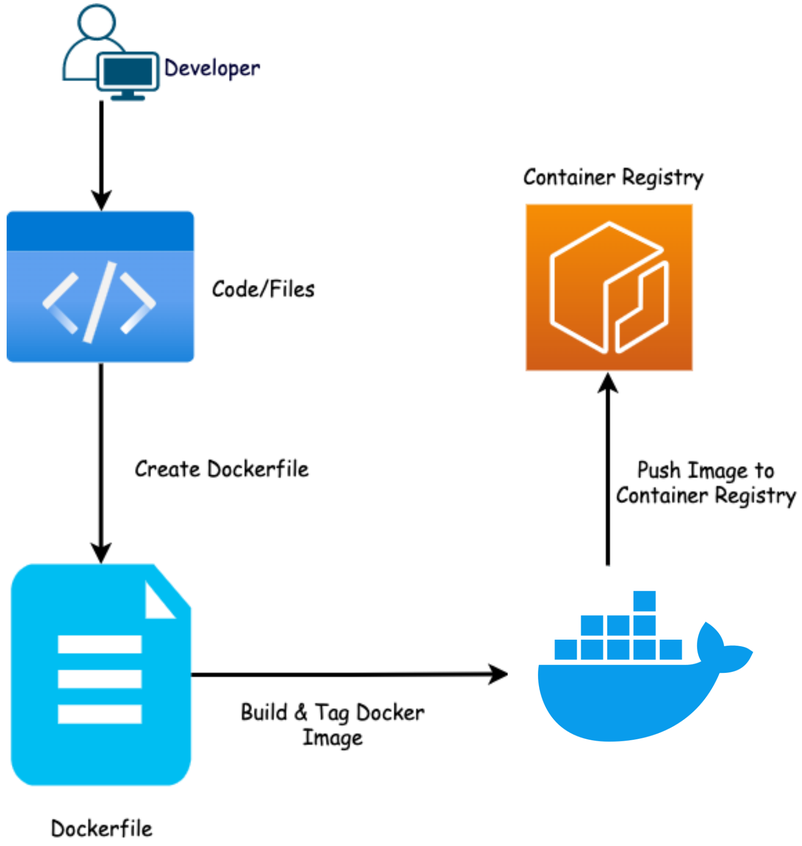
DOCKERIZATION



- Test your codebase locally
- Write a Dockerfile
- Build, run & test your container

1.3 AWS ECR

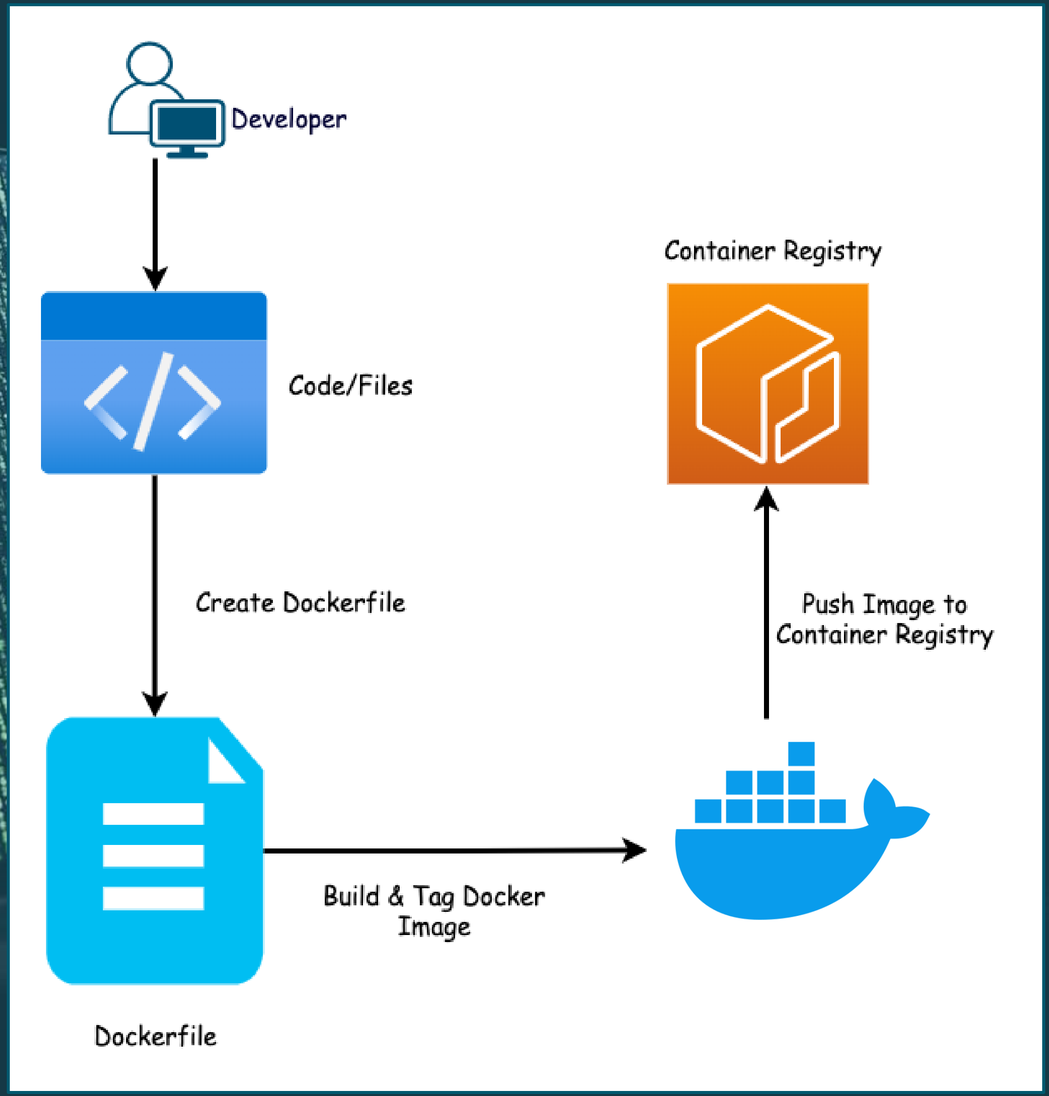
NATIVE CONTAINER REGISTRY



- Secure storage of images
- Seamless integration with other services
- Push & pull images using **AWS** CLI

Hands-on session 1: Building Docker Images

Infofarm
DATA CULTIVATORS



Infofarm
DATA CULTIVATORS

Break time

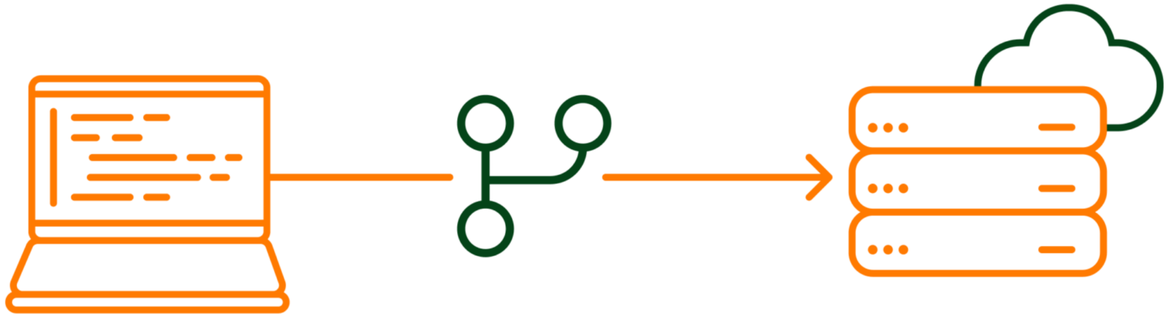
Grab a coffee ☕

2
Deploying infrastructure

Continuous Delivery (CD)

2.1 INFRASTRUCTURE AS CODE

IAC FOR AUTOMATION



- Use code to define infrastructure & resources
- Automate provisioning/ migrating & patching
- Version control for consistency & stability

2.2 USING TERRAFORM

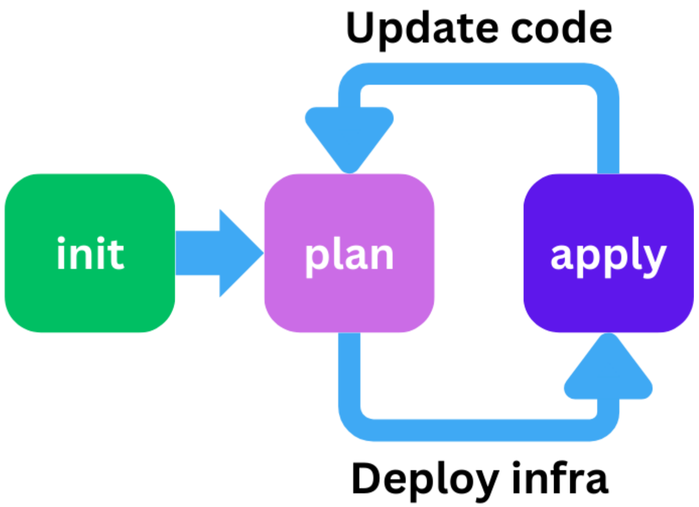
REUSABILITY IS KEY

```
variable "servers" {
  type = map(string)
  default = {
    server1 = "t2.micro",
    server2 = "t3.small",
    server3 = "t3.medium"
  }
}

resource "aws_instance" "server" {
  for_each = var.servers

  ami           = "ami-04b70fa74e45c3917"
  instance_type = each.value

  tags = {
    Name = each.key
  }
}
```



- Modules as reusable components
- Variables, data & resources
- terraform plan & terraform apply

Infofarm
DATA CULTIVATORS

2.3 AWS INFRA OVERVIEW

SERVICE GLOSSARY

Event manager

Artist/Line-Up

Security guard

Performance

- ELB for effective access & security
- ECS & ECR for managing & planning everything
- EC2 for instantiation of the application

Infofarm
DATA CULTIVATORS

2.3 AWS INFRA OVERVIEW

SERVICE GLOSSARY

AWS ECS

AWS ECR

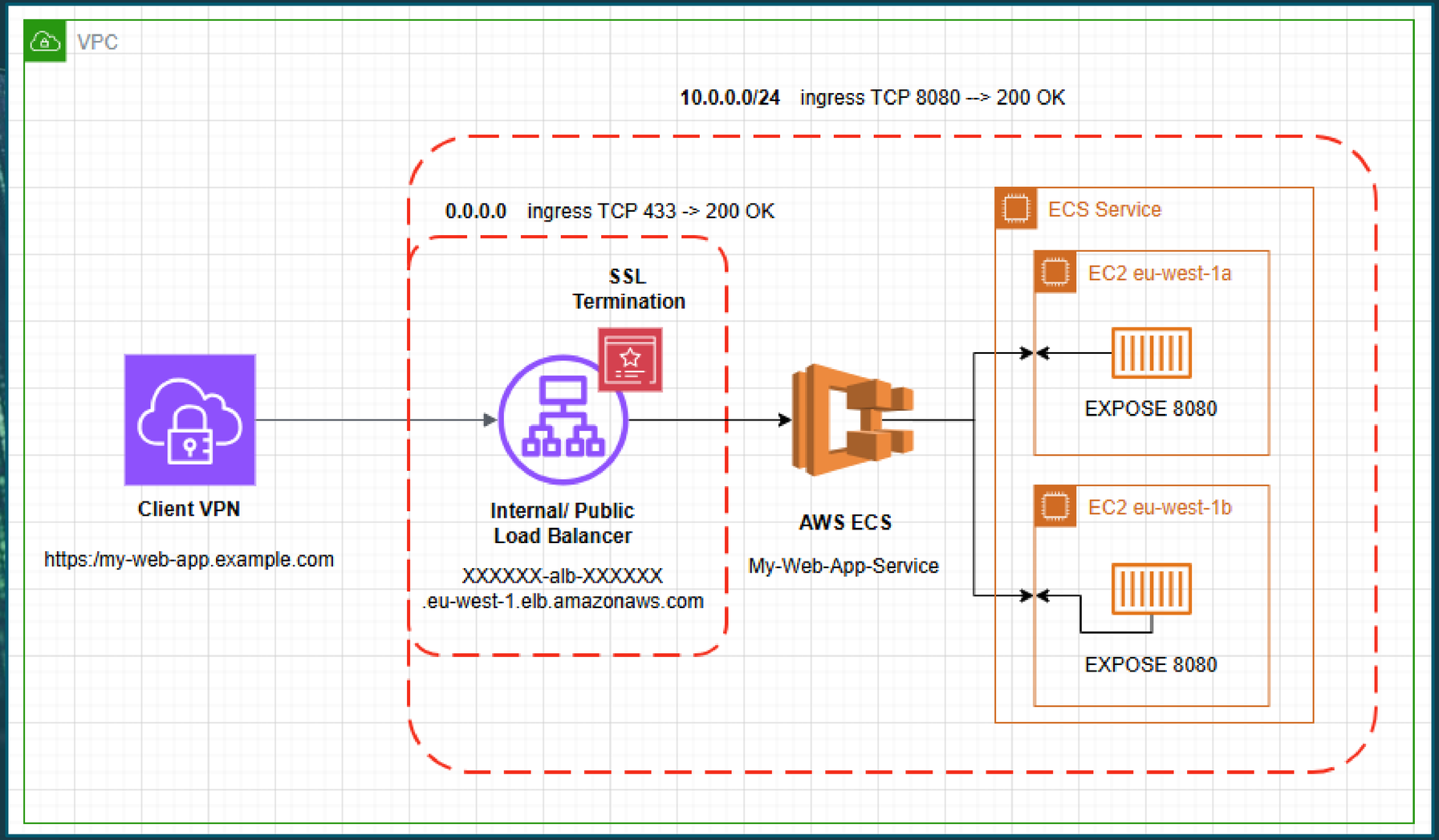
AWS ELB

AWS EC2

- ELB for load balancing & access to the machines
- ECS & ECR for scaling & deployment to machines
- EC2 for effective execution & hosting of source code

Hands-on session 2: Deploying with Terraform

Infofarm
DATA CULTIVATORS



Infofarm
DATA CULTIVATORS

Break time

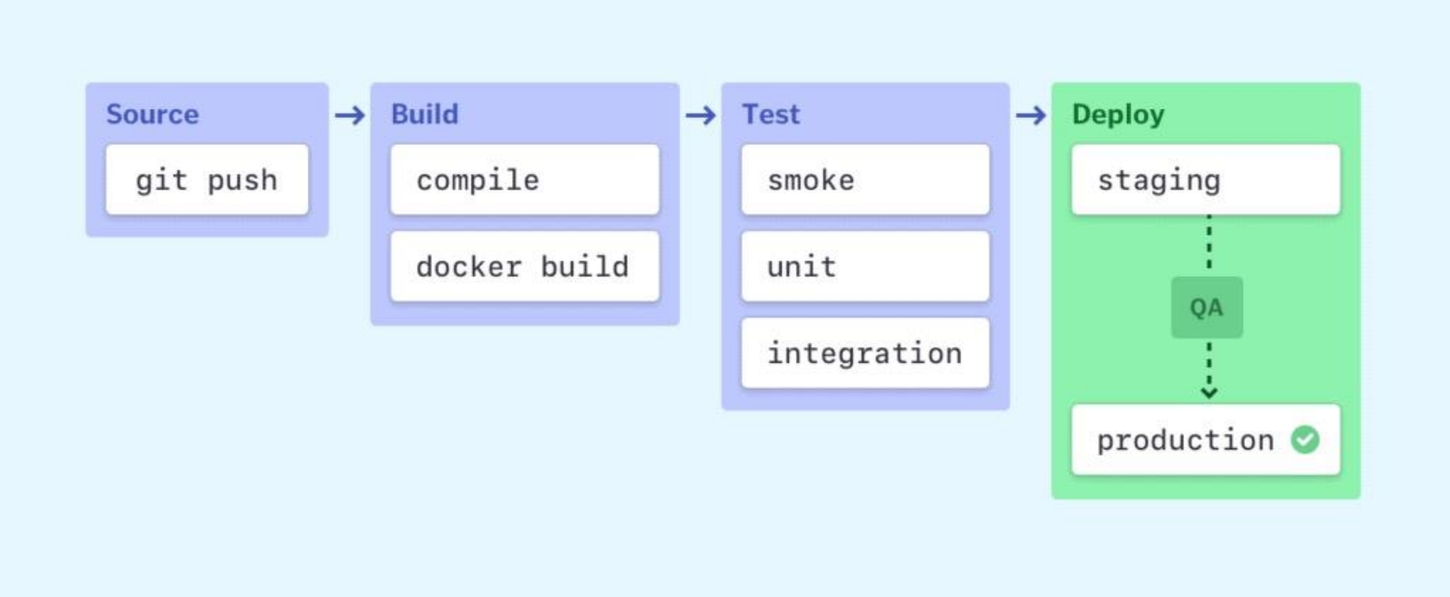
Grab a coffee ☕

3
Automating deployments

DevOps

3.1 CI/CD WORKFLOW

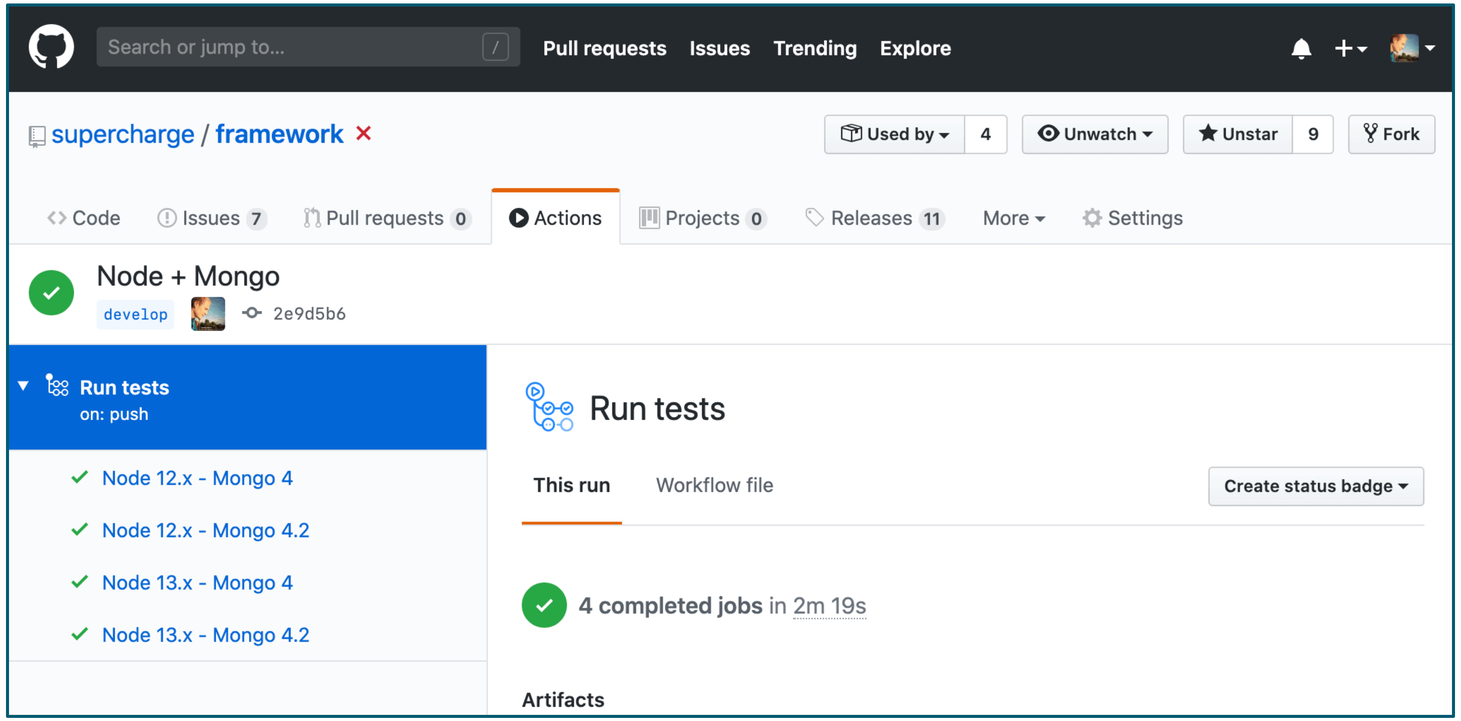
AUTOMATING THE WHOLE PROCESS



- Trigger builds, tests & deployments on push
- Standardize & streamline release process
- Ensure reliable, fast deployments

3.2 USING GITHUB ACTIONS

WRITING PIPELINES IN YML

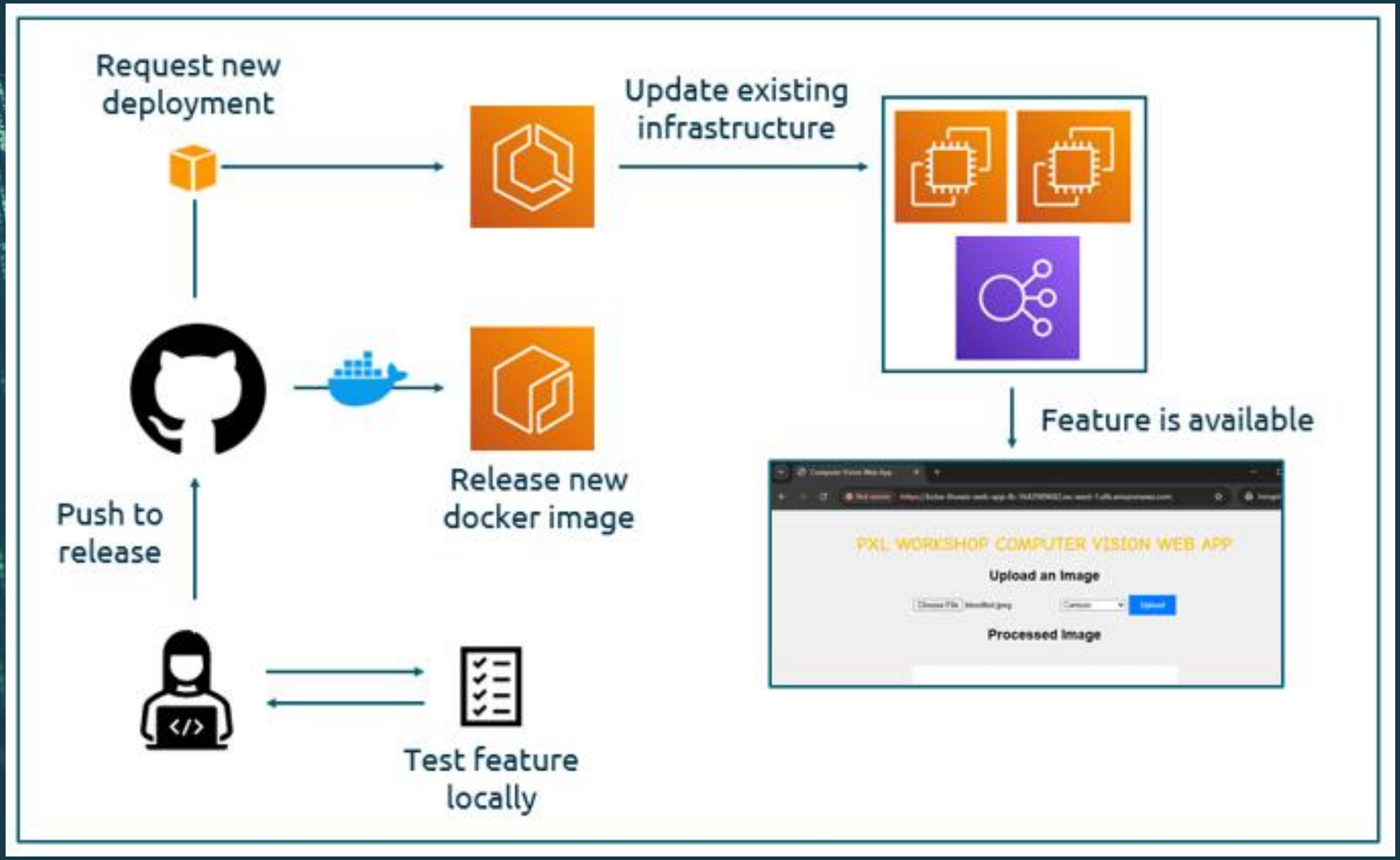


- Define in .github/workflows
- Use predefined actions
- Trigger on_push or on_pull_request

Infofarm
DATA CULTIVATORS

Demonstration: Pipelines with Github Actions

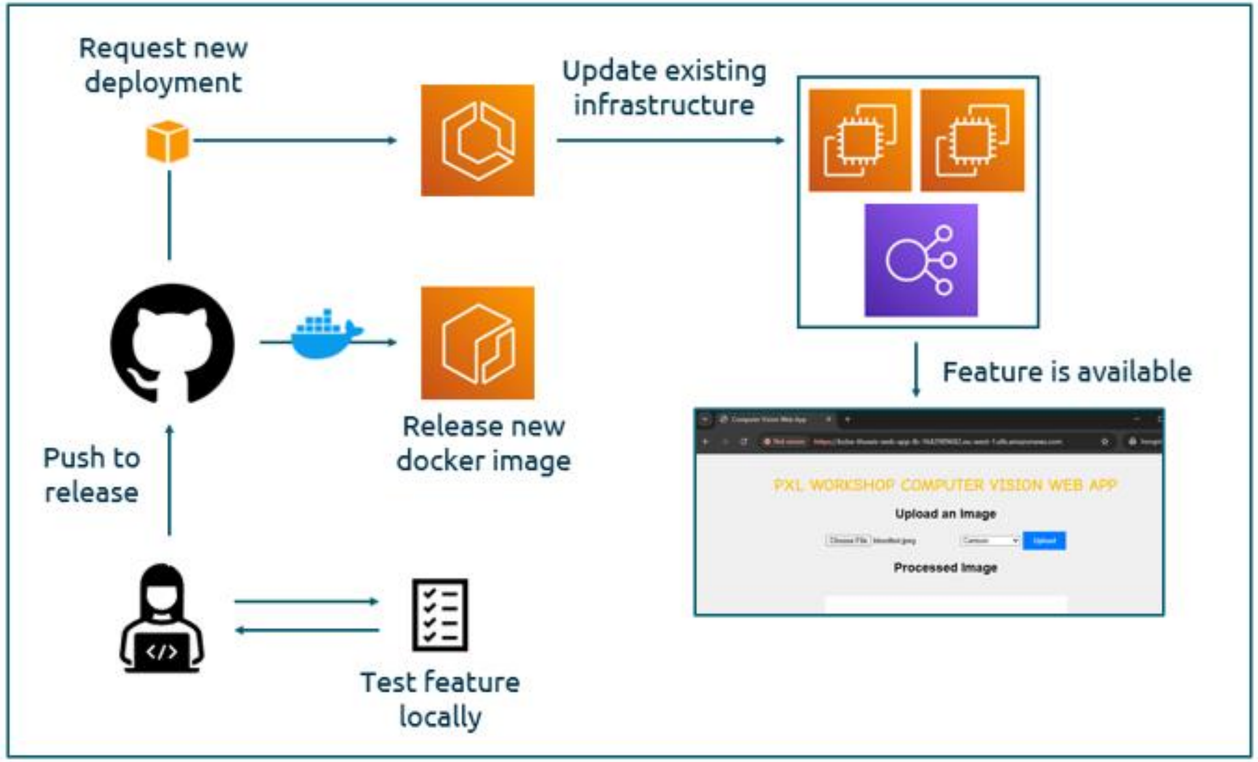
Infofarm
DATA CULTIVATORS



Infofarm
DATA CULTIVATORS

CONCLUSION

CI/CD FOR WEB APPS IN AWS



- Docker & Terraform for flexibility & modularity
- AWS for scalability, cost-efficiency & robustness
- Github Actions for a developer-centric process

Infofarm
DATA CULTIVATORS

Infofarm

DATA CULTIVATORS